Python & Algorithms for Coding Interviews

Section 2 — Data Structures Cheat Sheet

2.1 Arrays / Lists

What It Is

An array is a contiguous block of memory storing elements of the same type. Python's list is a dynamic array — it automatically resizes as you add elements. It is the most fundamental and frequently used data structure in coding interviews.

Intuition

Think of a list as a numbered row of boxes. You can instantly reach any box by its number (index). Adding to the end is fast. Inserting or removing from the middle requires shifting all boxes after that point.

Analogy

Mental Model	A row of lockers at school. Each locker has a number. You can open locker 5 directly without checking lockers 1–4. But if you insert a new locker in the middle, every locker after it must be renumbered and moved.

Internal Working

Python lists are implemented as arrays of pointers (object references). The list object stores a pointer to a block of memory containing pointers to each element. When the list runs out of space, Python allocates a larger block (roughly 1.125× the current size) and copies all pointers over. This is why append() is O(1) amortized but occasional appends cost O(n).

Python Usage

Python lists are used for almost everything — storing results, managing DFS/BFS queues, prefix sums, frequency arrays, sliding windows, and DP tables.

Code Example

```
nums = [1, 2, 3, 4, 5]

# Access by index — O(1)
print(nums[0])    # 1
print(nums[-1])   # 5  (last element)

# Slicing — O(k) where k = slice length
print(nums[1:4])  # [2, 3, 4]

# Append — O(1) amortized
nums.append(6)

# Insert — O(n)
nums.insert(2, 99)   # insert 99 at index 2

# Remove — O(n)
nums.remove(99)      # removes first occurrence

# Pop from end — O(1)
nums.pop()

# Pop from front — O(n) □ use deque instead
nums.pop(0)

# Check membership — O(n)
3 in nums

# 2D array (grid) — correct initialisation
grid = [[0] * cols for _ in range(rows)]
```

Time & Space Complexity

Operation	Time	Space
Access by index	O(1)	O(n)
Append (end)	O(1) amortized	
Pop (end)	O(1)	
Pop (front) / insert	O(n)	

Search (in)	O(n)	
Slice [i:j]	O(k)	O(k)
Sort	O(n log n)	O(n)

Interview Use Cases

- Two pointer problems — left/right pointers on same array
- Sliding window — maintain a window over contiguous elements
- Prefix sums — precompute cumulative values
- DP tables — store subproblem results
- BFS/DFS — store adjacency lists
- Frequency arrays — count characters or numbers

Real World Python Usage

Real World	Django ORM querysets return lists. NumPy arrays underpin all ML model training. FastAPI response bodies are often lists of Pydantic objects. Pandas DataFrames are built on NumPy arrays internally.

2.2 Hash Tables (dict and set)

What It Is

A hash table maps keys to values using a hash function. Python's dict and set are both hash tables. They provide O(1) average-case lookup, insertion, and deletion — making them the most powerful tool in interview problem solving.

Intuition

Imagine you have a million books and you want to find one instantly. Instead of searching shelf by shelf, you compute a 'code' from the book title and go directly to that shelf. The code is the hash, the shelf is the bucket.

Analogy

Mental Model	A library with a magic formula. Give any book title to the formula and it tells you exactly which shelf it is on — no searching needed. Hash tables work the same way.

Internal Working

Python's dict uses open addressing with a hash table. When you set d[key] = value, Python calls hash(key) to compute an integer index. It places the value at that index in an internal array. On lookup, the same hash is computed and the value is retrieved in O(1). Collisions (two keys hashing to the same index) are resolved by probing nearby slots.

Python Usage

Use dict for key-value mapping, counting, memoization, and graph adjacency. Use set for O(1) membership testing and deduplication.

Code Example

```
from collections import defaultdict, Counter

# Basic dict operations
d = {}
d['key'] = 'value'          # insert — O(1)
val = d.get('key', None)    # safe get with default
del d['key']                # delete — O(1)
'key' in d                  # membership — O(1)

# defaultdict — no KeyError for missing keys
freq = defaultdict(int)
for ch in 'hello':
    freq[ch] += 1

# Counter — instant frequency map
freq = Counter('mississippi')
# Counter({'s':4,'i':4,'p':2,'m':1})

# Set operations
seen = set()
seen.add(5)           # O(1)
5 in seen             # O(1)
seen.discard(5)       # O(1) — no error if missing

# Set math
a = {1, 2, 3}
b = {2, 3, 4}
a & b   # intersection: {2, 3}
a | b   # union:        {1, 2, 3, 4}
a - b   # difference:   {1}
```

Time & Space Complexity

Operation	Time	Space
Insert / Update	O(1) average	O(n)
Lookup / Delete	O(1) average	
Membership test	O(1) average	
Iteration	O(n)	
Worst case (collisions)	O(n)	

Interview Use Cases

- Frequency counting — count characters, words, or numbers
- Two Sum pattern — store complement values for O(1) lookup
- Memoization — cache recursive call results
- Grouping — group anagrams, group by property
- Graph adjacency list — defaultdict(list)
- Sliding window — track character counts in a window

Real World Python Usage

Real World	Python dict is used in caching systems (Redis-like in-memory caches), session storage in Django/Flask, JSON parsing (every JSON object becomes a dict), and configuration management. Sets are used in deduplication pipelines and bloom filter approximations.

2.3 Stacks

What It Is

A stack is a Last In First Out (LIFO) data structure. The last element you push onto the stack is the first one you pop off. Python implements stacks using a regular list — push with append(), pop with pop().

Intuition

Think of undo history in a text editor. Every action gets added to the top. When you press Ctrl+Z, the most recent action is undone first. You always interact with the top of the stack.

Analogy

Mental Model	A stack of dinner plates. You always place a new plate on top and always take from the top. You cannot take a plate from the middle without removing all the plates above it first.

Internal Working

A stack is conceptually simple — it is just a list where you only ever touch the right end. append() adds to the top (O(1)). pop() removes from the top (O(1)). Peeking at the top is done with stack[-1] — O(1). Python's list is perfect for this since append and pop from the right end are both O(1).

Python Usage

Python list used as stack: append() to push, pop() to pop, stack[-1] to peek.

Code Example

```
# Stack using Python list
stack = []

stack.append(1)     # push — O(1)
stack.append(2)
stack.append(3)

top = stack[-1]     # peek — O(1), no removal
val = stack.pop()   # pop  — O(1)

if not stack:       # check empty
    print('empty')

# Classic pattern — valid parentheses
def is_valid(s):
    stack = []
    pairs = {')':'(', ']':'[', '}':'{'}
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        elif stack and stack[-1] == pairs[ch]:
            stack.pop()
        else:
            return False
```

```
    return len(stack) == 0
```

Time & Space Complexity

Operation	Time	Space
Push (append)	O(1)	O(n)
Pop	O(1)	
Peek (top)	O(1)	
Search	O(n)	

Interview Use Cases

- Balanced parentheses — valid brackets problems
- Monotonic stack — next greater / smaller element
- DFS iterative — explicit stack replaces call stack
- Expression evaluation — infix to postfix, calculator
- Backtracking — undo moves in iterative solutions
- Call stack simulation — flatten nested structures

Real World Python Usage

Real World	Python's own call stack is a stack (as covered in Section 1). Django middleware uses a stack-based execution chain. Undo/redo systems in editors use two stacks. Browser history is a stack. Python's contextlib uses a stack-based context manager.

2.4 Queues

What It Is

A queue is a First In First Out (FIFO) data structure. The first element you enqueue is the first one you dequeue. Queues are fundamental to BFS algorithms. In Python, always use collections.deque — never a list — for queue operations.

Intuition

Think of a queue as a line at a ticket counter. The first person in line is served first. New people join at the back. Nobody cuts the line.

Analogy

Mental Model	A queue at a coffee shop. The first customer in line gets their coffee first. You always add new customers at the back and serve from the front.

Internal Working

A deque (double-ended queue) is implemented as a doubly-linked list of fixed-size blocks. This allows O(1) append and pop from both ends. A regular Python list has O(n) for pop(0) because all elements must shift left — never use list as a queue in interviews.

Python Usage

Use collections.deque for all queue operations. It supports appendleft(), append(), popleft(), pop() — all O(1).

Code Example

```
from collections import deque

queue = deque()

queue.append(1)      # enqueue — O(1)
queue.append(2)
queue.append(3)

front = queue[0]         # peek front — O(1)
val   = queue.popleft() # dequeue   — O(1)

# BFS template using queue
def bfs(graph, start):
    visited = set([start])
    queue   = deque([start])
    while queue:
        node = queue.popleft()
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
```

Time & Space Complexity

Operation	Time	Space
Enqueue (append)	O(1)	O(n)
Dequeue (popleft)	O(1)	
Peek front	O(1)	
list.pop(0) □	O(n)	

Interview Use Cases

- BFS — level-by-level tree and graph traversal
- Shortest path in unweighted graphs
- Multi-source BFS — multiple starting nodes simultaneously
- Level order tree traversal
- Sliding window using deque — monotonic queue
- Task scheduling simulation

Real World Python Usage

□ **Real World**	Python's asyncio event loop processes coroutines using a queue. Django Channels uses a message queue for WebSocket handling. Celery task queues distribute background jobs. Print spoolers and OS process schedulers use FIFO queues.

2.5 Deque (Double-Ended Queue)

What It Is

A deque allows O(1) insertions and deletions from both the front and the back. It is more powerful than both a stack and a queue, which only support one-ended operations. In Python, collections.deque is the standard implementation.

Intuition

A deque is like a double-ended tube. You can push and pop elements from either end with equal speed. This makes it perfect for algorithms that need both stack and queue behaviour — like the sliding window maximum.

Analogy

□ **Mental Model**	A train where passengers can board and exit from both the front and the rear door simultaneously — and both doors are equally fast.

Internal Working

CPython's deque is implemented as a doubly-linked list of fixed-size blocks (each block holds 64 elements). This gives O(1) operations at both ends without the need to shift elements. Random access in the middle (deque[i]) is O(n) — unlike lists which are O(1). Use deque when you need both ends, list when you need random access.

Python Usage

collections.deque with appendleft(), append(), popleft(), pop(). Use maxlen parameter to auto-discard old elements.

Code Example

```
from collections import deque

dq = deque([1, 2, 3])

dq.append(4)        # add to right  — O(1)
dq.appendleft(0)    # add to left   — O(1)
dq.pop()            # remove right  — O(1)
dq.popleft()        # remove left   — O(1)

dq[0]               # peek left     — O(1)
dq[-1]              # peek right    — O(1)
dq[2]               # random access — O(n) □

# maxlen — fixed-size sliding window
window = deque(maxlen=3)
for num in [1, 2, 3, 4, 5]:
    window.append(num)
# deque([3, 4, 5], maxlen=3)

# Monotonic deque — sliding window maximum
def max_sliding_window(nums, k):
    dq, result = deque(), []
    for i, num in enumerate(nums):
```

```
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if dq[0] < i - k + 1:
            dq.popleft()
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

Time & Space Complexity

Operation	Time	Space
append / appendleft	O(1)	O(n)
pop / popleft	O(1)	
Peek both ends	O(1)	
Random access [i]	O(n)	

Interview Use Cases

- Sliding window maximum — monotonic deque
- BFS queue — popleft is O(1)
- Palindrome check — compare from both ends
- LRU Cache — deque tracks usage order
- Sliding window problems needing index tracking

Real World Python Usage

Real World	Python's own asyncio uses deque internally for task scheduling. The LRU cache in functools is implemented with an ordered dict (which internally behaves like a deque). Browser tab history can use a deque structure.

2.6 Heaps (Priority Queues)

What It Is

A heap is a complete binary tree that satisfies the heap property: in a min-heap, every parent is smaller than or equal to its children. The minimum element is always at the root — accessible in O(1). Insertion and removal are O(log n). Python's heapq implements a min-heap.

Intuition

A heap is like a tournament bracket where the winner (smallest element) always bubbles to the top. Every time you remove the top winner, the bracket reorganises to find the next winner in O(log n) time.

Analogy

Mental Model	An airport boarding queue. First class passengers (highest priority = smallest number) always board first, no matter when they arrived. When a first class passenger boards, the next highest priority passenger immediately moves to the front.

Internal Working

A heap is stored as a flat array. For a node at index i, its left child is at 2i+1, right child at 2i+2, and parent at (i-1)//2. When you push a new element, it goes to the end (O(1)) and bubbles up (sifts up) by swapping with its parent until the heap property is restored — O(log n). When you pop the root, the last element replaces it and sifts down — O(log n).

Python Usage

heapq module. Python only provides min-heap. For max-heap, negate values when pushing and negate again when popping.

Code Example

```
import heapq

# Min-heap
heap = []
heapq.heappush(heap, 5)      # O(log n)
heapq.heappush(heap, 1)
heapq.heappush(heap, 3)

smallest = heap[0]           # peek   — O(1)
val = heapq.heappop(heap)    # pop    — O(log n) → returns 1

nums = [5, 1, 3, 2, 4]
heapq.heapify(nums)          # convert list in-place — O(n)
```

```
# Max-heap — negate values
max_heap = []
heapq.heappush(max_heap, -5)
heapq.heappush(max_heap, -1)
largest = -heapq.heappop(max_heap)   # 5

# Heap with (priority, value) tuples
tasks = []
heapq.heappush(tasks, (1, 'high priority'))
heapq.heappush(tasks, (3, 'low priority'))
heapq.heappush(tasks, (2, 'medium priority'))
priority, task = heapq.heappop(tasks)   # (1, 'high priority')

# K largest elements
heapq.nlargest(3, nums)    # O(n log k)
heapq.nsmallest(3, nums)   # O(n log k)
```

Time & Space Complexity

Operation	Time	Space
heappush	O(log n)	O(n)
heappop (min)	O(log n)	
Peek minimum	O(1)	
heapify	O(n)	
nlargest / nsmallest	O(n log k)	

Interview Use Cases

- Top K elements — K largest, K smallest, K most frequent
- Kth largest / smallest element
- Merge K sorted lists
- Dijkstra shortest path — min-heap on distances
- Task scheduling — process jobs by priority
- Median of data stream — two heaps technique

Real World Python Usage

Real World	Python's heapq powers task schedulers in asyncio. Celery uses priority queues for task dispatch. Database query optimisers use heap structures for top-k result processing. Operating system process schedulers use priority queues.

2.7 Linked Lists

What It Is

A linked list is a sequence of nodes where each node stores a value and a pointer to the next node. Unlike arrays, nodes are not stored contiguously in memory — each node lives wherever Python allocates it. Python has no built-in linked list, so you implement it with a Node class.

Intuition

A linked list is like a treasure hunt where each clue tells you the location of the next clue. You cannot skip ahead — you must follow the chain from the beginning. But inserting a new clue anywhere in the chain only requires updating two pointers, not moving everything else.

Analogy

Mental Model	A chain of train carriages. Each carriage is connected to the next. To add a new carriage in the middle, you just disconnect two carriages and reconnect with the new one in between — no other carriages need to move.

Internal Working

Each node holds data and a next pointer. The head pointer points to the first node. The last node's next pointer is None. Singly linked lists only have next. Doubly linked lists have both next and prev, enabling O(1) deletion when you have a reference to the node. Python's collections.deque is actually a doubly linked list of arrays internally.

Python Usage

Implement with a custom Node class. Python's deque is backed by a linked list. Use dummy head nodes to simplify edge cases.

Code Example

```
# Node definition
class ListNode:
    def __init__(self, val=0, next=None):
        self.val  = val
        self.next = next
```

```
# Build:  1 -> 2 -> 3 -> None
head = ListNode(1)
head.next = ListNode(2)
head.next.next = ListNode(3)

# Traverse
node = head
while node:
    print(node.val)
    node = node.next

# Reverse a linked list — classic interview problem
def reverse(head):
    prev, curr = None, head
    while curr:
        nxt       = curr.next
        curr.next = prev
        prev      = curr
        curr      = nxt
    return prev

# Dummy head — simplifies insertion/deletion edge cases
dummy = ListNode(0)
dummy.next = head

# Fast and slow pointer — find middle
def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

Time & Space Complexity

Operation	Time	Space
Access by index	O(n)	O(n)
Insert at head	O(1)	
Insert at tail	O(n) / O(1) with tail ptr	
Insert at position	O(n)	
Delete (with node ref)	O(1) doubly linked	
Search	O(n)	

Interview Use Cases

- Reverse a linked list — iterative and recursive

- Detect cycle — fast and slow pointers
- Find middle node — fast and slow pointers
- Merge two sorted lists
- Remove Nth node from end
- LRU Cache — doubly linked list + hash map

Real World Python Usage

Real World	Python's collections.deque is a doubly linked list internally. LRU cache implementations use a doubly linked list paired with a hash map. Memory allocators use free-list linked lists. File systems use linked structures for block chains.

2.8 Trees

What It Is

A tree is a hierarchical data structure with a root node at the top. Every node has zero or more child nodes. A binary tree is the most common type in interviews — each node has at most two children (left and right). Trees are acyclic, connected graphs with a designated root.

Intuition

A tree is like a company org chart. The CEO is the root. Each manager (node) has direct reports (children). Leaf nodes are employees with no direct reports. The depth of a node is how many levels it is from the root. The height of the tree is the longest path from root to any leaf.

Analogy

Mental Model	A family tree. The great-grandparent is the root. Their children, grandchildren, and great-grandchildren form branches. Leaf nodes are the youngest generation with no children.

Internal Working

Trees are implemented with nodes, each containing a value and references to children. Binary trees have left and right child references. The height of a balanced binary tree with n nodes is

O(log n). A skewed (unbalanced) tree can degrade to height O(n) — effectively a linked list. DFS and BFS are the two fundamental ways to traverse a tree.

Python Usage

Implement with TreeNode class. Use recursion for DFS traversals. Use deque for level-order BFS traversal.

Code Example

```
# TreeNode definition (standard in LeetCode)
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val   = val
        self.left  = left
        self.right = right

# Inorder:   Left → Root → Right  (gives sorted order in BST)
def inorder(root):
    if not root: return []
    return inorder(root.left) + [root.val] + inorder(root.right)

# Preorder:  Root → Left → Right  (good for copying a tree)
def preorder(root):
    if not root: return []
    return [root.val] + preorder(root.left) + preorder(root.right)

# Postorder: Left → Right → Root  (good for deleting a tree)
def postorder(root):
    if not root: return []
    return postorder(root.left) + postorder(root.right) + [root.val]

# Level-order BFS traversal
from collections import deque
def level_order(root):
    if not root: return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:  queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result

# Tree height
def height(root):
    if not root: return 0
    return 1 + max(height(root.left), height(root.right))
```

Time & Space Complexity

Operation	Time	Space
DFS traversal	O(n)	O(h) stack
BFS traversal	O(n)	O(w) queue
Height / depth	O(n)	O(h)
h = balanced tree	O(log n)	
h = skewed tree	O(n)	

Interview Use Cases

- Tree traversals — inorder, preorder, postorder, level-order
- Lowest Common Ancestor (LCA)
- Maximum path sum
- Diameter of binary tree
- Symmetric / mirror tree check
- Serialise and deserialise a tree

Real World Python Usage

Real World	File systems are trees — directories are internal nodes, files are leaves. HTML/XML DOM is a tree structure. Python's AST (Abstract Syntax Tree) represents your code as a tree. Decision trees in machine learning are literal binary trees.

2.9 Binary Search Trees (BST)

What It Is

A Binary Search Tree is a binary tree with a key property: for every node, all values in the left subtree are smaller, and all values in the right subtree are larger. This property enables O(log n) search, insert, and delete in a balanced tree.

Intuition

A BST is like a binary decision game. At each node, you ask: is my target smaller or larger than this value? If smaller, go left. If larger, go right. Each step eliminates half the remaining possibilities — just like binary search on a sorted array.

Analogy

Mental Model	A library sorted alphabetically on shelves arranged in a tree. At the middle shelf you find 'M'. Searching for 'Apple'? Go left. Searching for 'Zebra'? Go right. Each decision halves your search space.

Internal Working

The BST property makes inorder traversal produce a sorted sequence — this is the most important property to remember. A balanced BST (like AVL tree or Red-Black tree) guarantees O(log n) height. Python has no built-in BST. The sortedcontainers library provides SortedList. In interviews, you implement BST logic manually.

Python Usage

Implement manually in interviews. Use sortedcontainers.SortedList for production code. Inorder traversal of BST gives sorted output.

Code Example

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

# Search — O(log n) balanced, O(n) worst
def search(root, target):
    if not root or root.val == target:
        return root
    if target < root.val:
        return search(root.left, target)
    return search(root.right, target)

# Insert — O(log n) balanced
def insert(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left  = insert(root.left, val)
    else:
        root.right = insert(root.right, val)
    return root
```

```
# Validate BST — pass min/max bounds
def is_valid_bst(root, low=float('-inf'), high=float('inf')):
    if not root: return True
    if not (low < root.val < high): return False
    return (is_valid_bst(root.left,  low, root.val) and
            is_valid_bst(root.right, root.val, high))

# Inorder traversal gives sorted sequence
def inorder(root):
    if not root: return []
    return inorder(root.left) + [root.val] + inorder(root.right)
```

Time & Space Complexity

Operation	Time	Space
Search / Insert / Delete (balanced)	O(log n)	O(h)
Search / Insert / Delete (skewed)	O(n)	
Inorder traversal	O(n)	O(h)
Min / Max	O(h)	

Interview Use Cases

- Validate a BST — check bounds recursively
- Find Kth smallest/largest element in BST
- LCA in a BST
- Convert sorted array to balanced BST
- BST iterator
- Range sum query — sum all nodes in [L, R]

Real World Python Usage

Real World	Database indexes (B-trees, which are generalised BSTs) power every SQL query. Python's sortedcontainers.SortedList is used in competitive programming and data science. Git commit history is a tree. Event-driven schedulers use BST-like structures for time-ordered event dispatch.

2.10 Graphs

What It Is

A graph is a collection of nodes (vertices) connected by edges. Unlike trees, graphs can have cycles, multiple paths between nodes, and disconnected components. Graphs model real-world networks: roads, social connections, dependencies, and more. They are the most complex and most rewarding data structure in interviews.

Intuition

A graph is like a city road map. Every intersection is a node. Every road is an edge. A directed graph (digraph) means roads can be one-way. A weighted graph means roads have distances. BFS finds the shortest path. DFS explores all paths.

Analogy

Mental Model	A city metro map. Each station is a node. Each track between stations is an edge. Weighted edges represent travel time. Directed edges represent one-way routes. Finding the fastest route from A to B is shortest path. Checking all reachable stations from A is graph traversal.

Internal Working

Graphs are stored as adjacency lists (dict of lists) for sparse graphs or adjacency matrices (2D array) for dense graphs. Adjacency lists use O(V + E) space — far more efficient than adjacency matrices ($O(V^2)$) for most real-world graphs. BFS uses a queue and explores level by level. DFS uses a stack (or recursion) and explores as deep as possible before backtracking.

Python Usage

Use defaultdict(list) for adjacency list. Use a set for visited nodes. BFS with deque, DFS with recursion or explicit stack.

Code Example

```
from collections import defaultdict, deque

# Build adjacency list — undirected graph
graph = defaultdict(list)
edges = [(0,1),(0,2),(1,3),(2,3),(3,4)]
for u, v in edges:
    graph[u].append(v)
```

```
    graph[v].append(u)   # omit for directed graph

# BFS — shortest path, level-order
def bfs(start, target):
    visited = set([start])
    queue   = deque([(start, 0)])   # (node, distance)
    while queue:
        node, dist = queue.popleft()
        if node == target: return dist
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                queue.append((nb, dist + 1))
    return -1

# DFS — explore all paths, detect cycles
def dfs(node, visited):
    visited.add(node)
    for nb in graph[node]:
        if nb not in visited:
            dfs(nb, visited)

# Count connected components
def count_components(n, edges):
    graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
    visited, count = set(), 0
    for node in range(n):
        if node not in visited:
            dfs(node, visited)
            count += 1
    return count
```

Time & Space Complexity

Operation	Time	Space
BFS / DFS traversal	O(V + E)	O(V)
Adjacency list space		O(V + E)
Adjacency matrix space		$O(V^2)$
Edge lookup (adj list)	O(degree)	
Edge lookup (adj matrix)	O(1)	

Interview Use Cases

- Number of islands — DFS/BFS on a matrix grid
- Shortest path in unweighted graph — BFS

- Detect cycle — directed and undirected
- Connected components — DFS/Union Find
- Bipartite graph check — BFS colouring
- Clone a graph

Real World Python Usage

Real World	Google Maps routing is graph shortest-path. Social network friend suggestions use graph BFS. Python's import system resolves module dependencies using a DAG. RAG pipelines in AI use knowledge graphs. Network packet routing uses graph algorithms.

2.11 Tries (Prefix Trees)

What It Is

A trie (pronounced 'try') is a tree-shaped data structure optimised for storing and searching strings by their prefixes. Each node represents one character. A path from the root to a node spells out a prefix. A path from root to an end node spells a complete word. Insert, search, and prefix-search all run in O(L) time where L is the length of the word.

Intuition

A trie is like an autocomplete system. When you type 'app' into Google, it instantly shows 'apple', 'application', 'appointment'. All words sharing the prefix 'app' share the same branch of the trie — no repeated character storage.

Analogy

Mental Model	A phone keypad word tree. Every time you press a letter, you go one level deeper in the tree. All words that share the same starting letters share the same path from the root. The trie 'fans out' as words diverge.

Internal Working

Each TrieNode stores a dictionary of children (one per character) and an is_end flag marking the end of a complete word. Inserting a word of length L visits or creates L nodes — O(L).

Searching works the same way. The total space is O(ALPHABET_SIZE × N × L) in the worst case, but shared prefixes save significant space in practice.

Python Usage

Implement with a TrieNode class containing a children dict and is_end boolean flag.

Code Example

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end   = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end

    def starts_with(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return True

# Usage
trie = Trie()
trie.insert('apple')
trie.search('apple')      # True
trie.search('app')        # False — not a complete word
trie.starts_with('app')   # True — valid prefix
```

Time & Space Complexity

Operation	Time	Space
Insert	O(L)	O(L)
Search	O(L)	O(1)
Prefix search	O(L)	O(1)
Space (all words)	O(ALPHABET × N × L)	

Interview Use Cases

- Implement Trie — insert, search, startsWith
- Word Search II — find all words in a grid using trie
- Replace words — replace words with their root prefix
- Design search autocomplete system
- Longest word in dictionary
- Map sum pairs — sum values of keys with given prefix

Real World Python Usage

Real World	Search engines use tries for autocomplete suggestions. IDEs use tries for code completion. IP routing tables use a form of trie (Patricia trie) for prefix matching. Spell checkers use tries to find similar words. DNS resolution uses trie-like prefix matching.

2.12 Data Structure Comparison Table

Use this table to quickly decide which data structure to use when you read a problem.

Data Structure	Best For	Key Operation	Python
Array / List	Indexed access, iteration	O(1) index access	list
Hash Table	Key-value, counting, fast lookup	O(1) get/set	dict, set
Stack	LIFO, undo, DFS, brackets	O(1) push/pop	list (append/pop)
Queue	FIFO, BFS, level-order	O(1) enqueue/dequeue	deque
Deque	Both ends, sliding window	O(1) both ends	collections.deque

Heap	Top K, priority, scheduling	O(log n) push/pop	heapq
Linked List	Insert/delete middle, LRU	O(1) insert with ref	custom Node class
Tree	Hierarchical data, traversals	O(h) traversal	custom TreeNode
BST	Sorted data, range queries	O(log n) search	sortedcontainers
Graph	Networks, paths, components	O(V+E) BFS/DFS	defaultdict(list)
Trie	Prefix search, autocomplete	O(L) insert/search	custom TrieNode

Section 2 — Quick Revision Cheat Sheet

Data Structure	Key Fact to Remember
List	Dynamic array — O(1) append, O(n) insert/delete middle, O(n) pop(0)
dict / set	Hash table — O(1) avg get/set/delete/in. Only hashable keys.
Stack	LIFO — list.append() push, list.pop() pop, list[-1] peek
Queue	FIFO — always use deque, never list (pop(0) is O(n))
Deque	O(1) both ends — use for BFS, sliding window, monotonic queue
Heap	Min-heap via heapq — negate values for max-heap
Linked List	O(1) insert at head — use dummy node to simplify edge cases
Tree	Inorder=sorted(BST), BFS=level order, DFS=recursion
BST	Inorder gives sorted output — validate with min/max bounds
Graph	defaultdict(list) for adjacency list — BFS=shortest path, DFS=explore all
Trie	children dict + is_end flag — O(L) for all ops where L = word length

End of Section 2. Continue with Section 3 — Core Algorithmic Patterns.